

1. How It Works

1.1 Leader-Follower

One leader (node0) accepts all writes, replicates to 4 followers sequentially with simulated delays (200ms between sends, 100ms on each follower). Reads go to any node; R=1 reads locally, R>1 gathers from peers (50ms delay each).

1.2 Leaderless

Any node can accept writes and becomes the Write Coordinator for that request. Replication works the same way as leader mode, just without a fixed leader.

1.3 W/R Configs

Config	W	R	W+R	Consistent?	Tradeoff
W=5, R=1	5	1	6 > 5	Yes	Slow writes, instant reads
W=1, R=5	1	5	6 > 5	Yes*	Fast writes, slow reads
W=3, R=3	3	3	6 > 5	Yes	Balanced
Leaderless	5	1	6 > 5	Yes	Slow writes, instant reads

**W=1 R=5 showed stale reads since the write returns before replication finishes.*

1.4 Version Numbers

Each key has an incrementing version; replicas only accept higher versions. Load tester tracks max version per key to detect stale reads.

2. N/R/W Implementation Details

2.1 Leader W=5, R=1

Write: Leader saves locally, then POSTs to each follower sequentially (200ms gap, 100ms follower delay). Blocks until all 4 ack. ~1200ms total.

Read: Local map lookup. ~3ms.

2.2 Leader W=1, R=5

Write: Leader saves locally and returns 201 immediately. Goroutine replicates in background. ~1ms client latency.

Read: Queries all 5 nodes in parallel (50ms peer delay), returns highest version. ~54ms.

2.3 Leader W=3, R=3

Write: Leader saves locally + waits for 2 followers (quorum of 3). Remaining 2 replicated in background. ~600ms.

Read: Queries peers in parallel, needs 3 total responses, returns highest version. ~55ms.

2.4 Leaderless W=5, R=1

Write: Receiving node becomes coordinator, replicates to all 4 others sequentially, waits for all. ~1250ms.

Read: Local map lookup. ~4ms.

3. Load Test Results

10 users, 60 seconds per run, 10-key pool for temporal locality.

3.1 Summary

Config	Ratio	Read Avg	Read p50	Read p99	Write Avg	Write p50	Write p99	Stale	Total Reqs
Leader W=5 R=1	1/99	3.1ms	3ms	8ms	1211ms	1200ms	1200ms	0	11,728
Leader W=5 R=1	10/90	2.9ms	3ms	7ms	1211ms	1200ms	1200ms	0	3,598
Leader W=5 R=1	50/50	3.2ms	3ms	9ms	1212ms	1200ms	1200ms	0	889
Leader W=5 R=1	90/10	6.0ms	6ms	17ms	1212ms	1200ms	1200ms	0	510
Leader W=1 R=5	1/99	54.2ms	54ms	60ms	40ms	45ms	52ms	0	6,670
Leader W=1 R=5	10/90	53.8ms	53ms	59ms	23ms	7ms	50ms	0	6,957
Leader W=1 R=5	50/50	53.9ms	54ms	59ms	46ms	45ms	52ms	5	7,018
Leader W=1 R=5	90/10	53.6ms	53ms	60ms	46ms	45ms	52ms	1	7,347
Leader W=3 R=3	1/99	55.1ms	55ms	62ms	608ms	610ms	620ms	0	6,091
Leader W=3 R=3	10/90	54.6ms	54ms	60ms	608ms	610ms	630ms	0	4,134
Leader W=3 R=3	50/50	54.6ms	54ms	61ms	608ms	610ms	610ms	0	1,672
Leader W=3 R=3	90/10	56.8ms	55ms	80ms	608ms	610ms	620ms	0	1,020
Leaderless W=5 R=1	1/99	4.6ms	4ms	11ms	1252ms	1300ms	1300ms	0	10,864
Leaderless W=5 R=10	10/90	3.4ms	3ms	8ms	1248ms	1300ms	1300ms	0	3,583
Leaderless W=5 R=50	50/50	3.4ms	3ms	10ms	1231ms	1200ms	1300ms	0	893
Leaderless W=5 R=90	90/10	4.0ms	4ms	11ms	1216ms	1200ms	1300ms	0	512

3.2 Throughput (requests/sec)

Config	Ratio	Read req/s	Write req/s	Total req/s
Leader W=5 R=1	1/99	195.8	2.0	197.8
Leader W=5 R=1	10/90	54.3	6.2	60.5
Leader W=5 R=1	50/50	7.3	7.6	15.0
Leader W=5 R=1	90/10	0.8	7.9	8.6
Leader W=1 R=5	1/99	111.3	1.3	112.6
Leader W=1 R=5	10/90	106.0	11.4	117.3
Leader W=1 R=5	50/50	59.5	58.9	118.4
Leader W=1 R=5	90/10	12.3	111.5	123.8

Leader W=3 R=3	1/99	101.4	1.4	102.7
Leader W=3 R=3	10/90	63.1	6.6	69.7
Leader W=3 R=3	50/50	14.9	13.2	28.2
Leader W=3 R=3	90/10	2.1	15.0	17.1
Leaderless W=5 R=1	1/99	181.2	2.1	183.4
Leaderless W=5 R=1	10/90	54.2	6.0	60.2
Leaderless W=5 R=1	50/50	7.5	7.5	15.0
Leaderless W=5 R=1	90/10	0.8	7.9	8.6

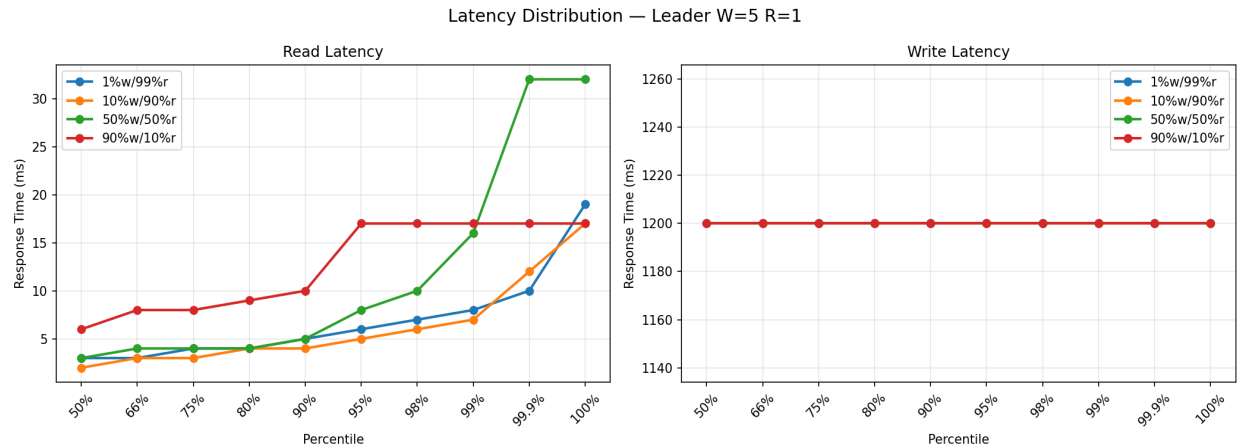
3.3 Stale Reads

Config	1/99	10/90	50/50	90/10	Total
Leader W=5 R=1	0	0	0	0	0
Leader W=1 R=5	0	0	5	1	6
Leader W=3 R=3	0	0	0	0	0
Leaderless W=5 R=1	0	0	0	0	0

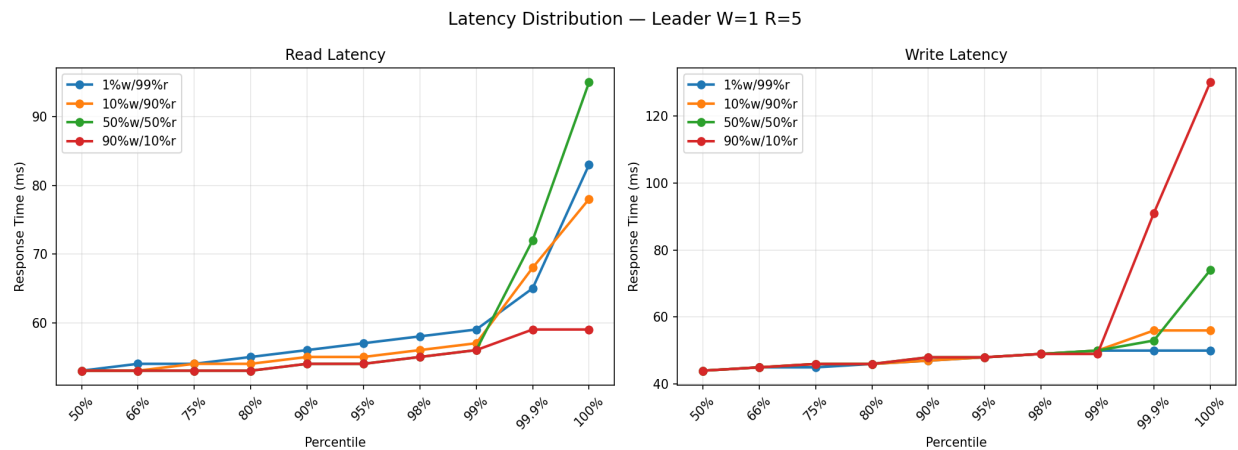
4. Graphs

4.1 Latency Distributions

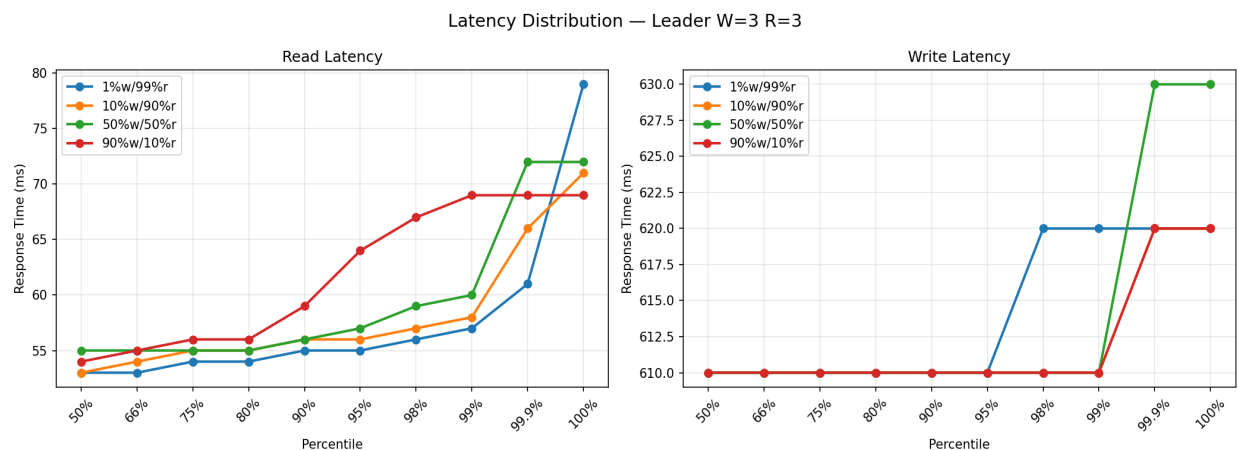
Leader W=5, R=1:



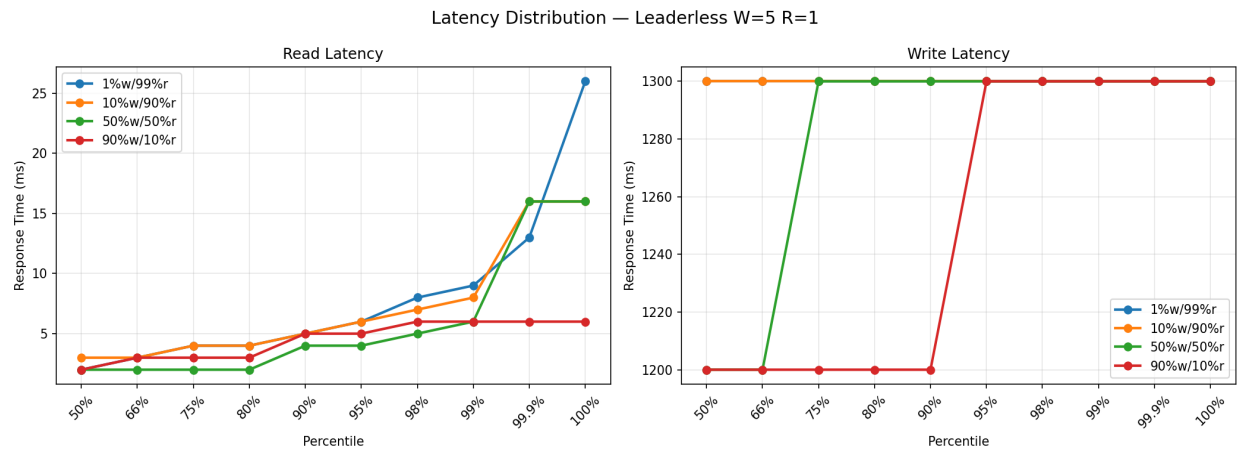
Leader W=1, R=5:



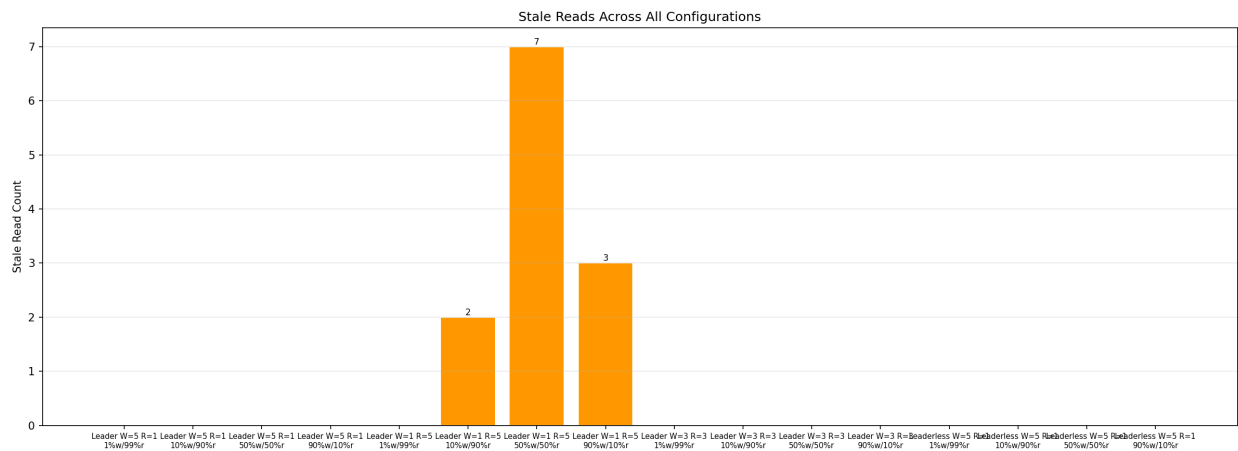
Leader W=3, R=3:



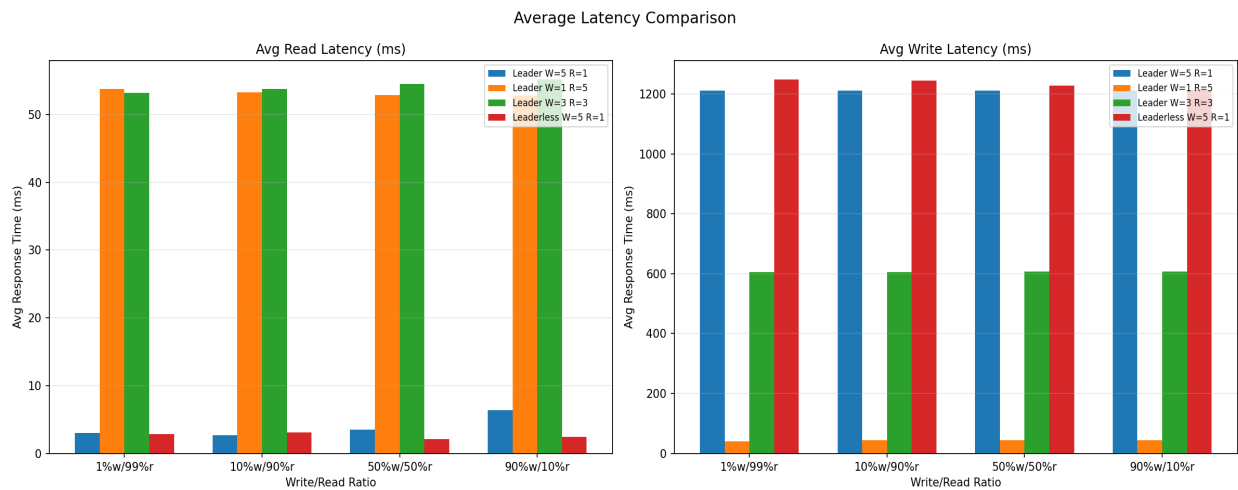
Leaderless W=5, R=1:



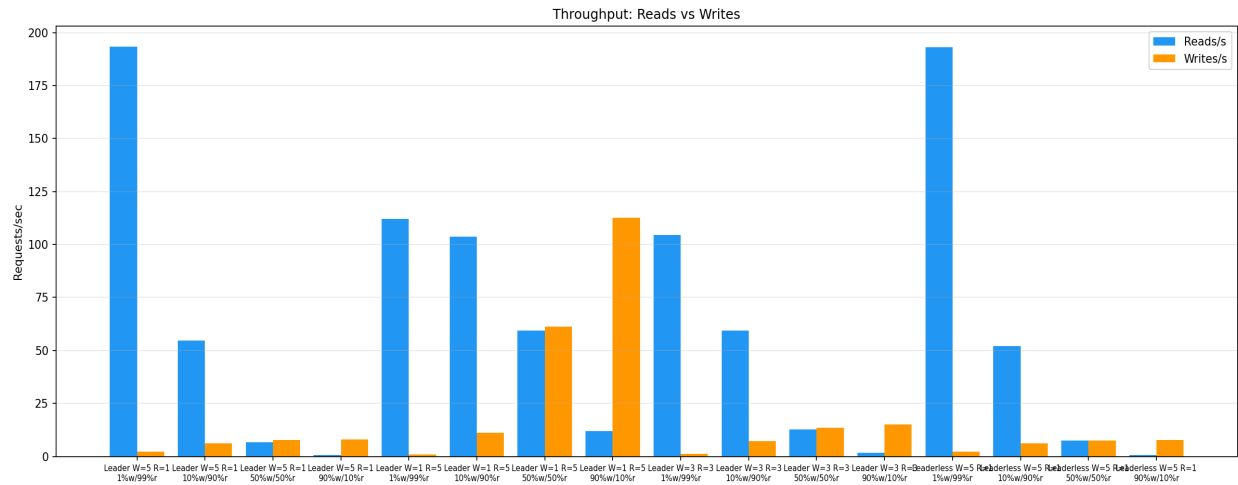
4.2 Stale Reads



4.3 Average Latency Comparison

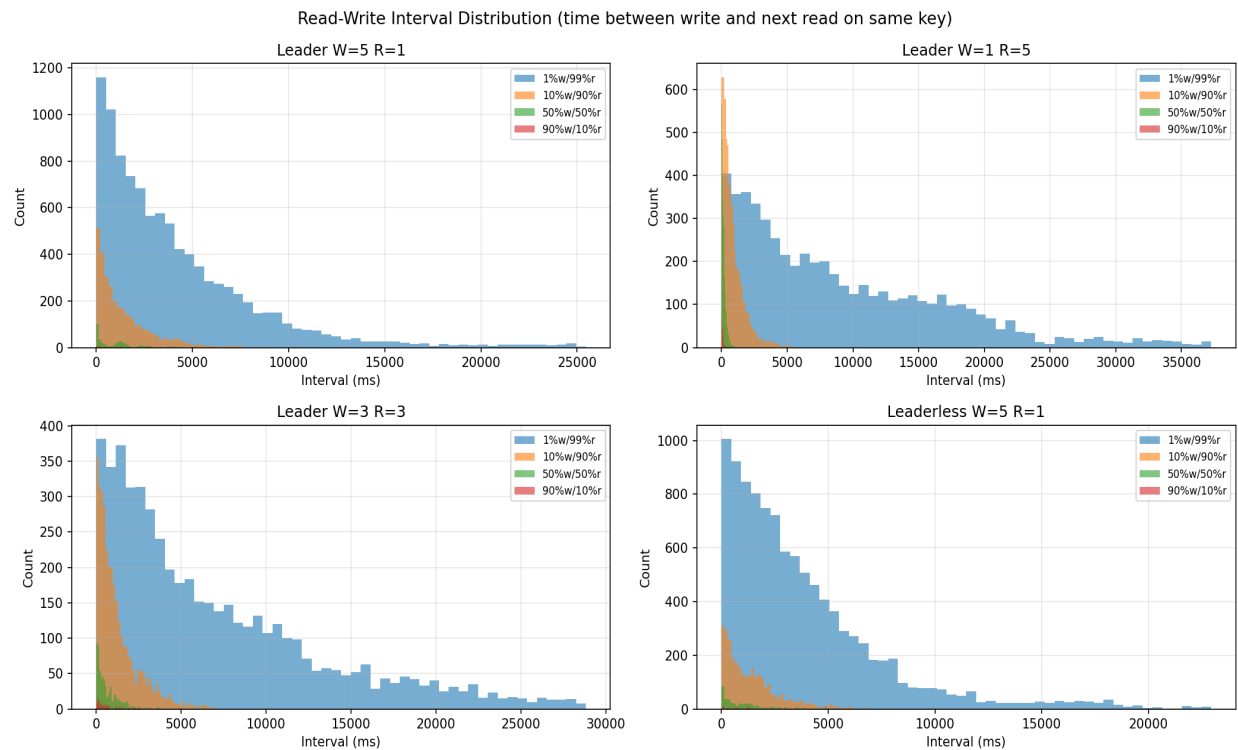


4.4 Throughput



4.5 Read-Write Interval Distribution

Time gap between a write and the next read to the same key. Small intervals mean the load tester is reading keys shortly after writing them, which is what we need to catch staleness.



5. Analysis

Leader W=5, R=1

- Writes fixed at ~1211ms (4 x 300ms sequential replication). Reads ~3ms (local lookup).
- 0 stale reads; all nodes updated before client gets 201.
- Throughput drops 23x from read-heavy to write-heavy (198 req/s to 8.5 req/s) because writes block for 1.2s.
- Best for read-heavy workloads like product catalogs.

Leader W=1, R=5

- Writes ~23-46ms (local save, background replication). Reads ~54ms (50ms peer sleep floor).
- 6 stale reads at higher write ratios; async replication races with the read gather.
- Throughput stays stable across ratios (~112-124 req/s) since both reads and writes are cheap.
- Best for write-heavy or balanced workloads.

Leader W=3, R=3

- Writes ~608ms (half of W=5; only needs 2 followers). Reads ~55ms (same 50ms floor).
- 0 stale reads; $W+R=6 > N=5$ guarantees quorum overlap.
- Good middle ground when you need consistency without 1.2s writes.

Leaderless W=5, R=1

- Similar to Leader W=5 R=1 but writes ~30-40ms slower (coordinator varies, less connection reuse).
- 0 stale reads; $W=N$ ensures full replication before ack.
- Trade-off vs leader: no single point of failure, but slightly worse write latency.

Test generator: 10 Locust users pick from a pool of 10 keys randomly. Small pool means reads and writes frequently hit the same key within short time windows.